Reparto del código — 4 personas, última semana

Cada persona se queda con **un bloque completo**: backend, pantallas y tests unitarios de su parte. La carga es pareja (unas 5 funcionalidades cada uno), y así cada uno puede explicar su parte de punta a punta en la defensa.

Persona 1 — Prestador: actividades y horarios

- Completar `Actividad`: título, descripción, `estado` como enum (ACTIVA / BAJA), relación con `Departamento`.
- Alta de actividad, validando que el emprendimiento esté **ACEPTADO** (RNE #8), con categorías y equipamiento.
- Editar actividad (bloqueada si tiene reservas, RNE #15) y baja lógica (RNE #9).
- Cargar horario con su política de cancelación; cancelar horario.
- Panel del prestador: “Mis actividades”, “Reservas recibidas” (CU-P11) y “Comentarios recibidos” (CU-P12, solo lectura).
- Editar datos del emprendimiento, incluida la descripción (CU-P05).
- **Tests:** crear con emprendimiento pendiente → rechaza · editar con reservas → rechaza · cupos disponibles = cupos totales al crear.

Persona 2 — Turista: explorar y reservar

- Entidades `Reserva` (+ enum PENDIENTE_PAGO / CONFIRMADA / CANCELADA) y `Participante`.
- Página pública **explorar actividades**, sin login: filtros (departamento, tipo, fecha, precio) + búsqueda por palabra clave. Solo actividades de emprendimientos aceptados.
- **Detalle de actividad**: datos, equipamiento, categorías, próximos horarios con cupos, valoración promedio.
- **Reservar**: cantidad de cupos + datos de cada participante + observación. Descuento **atómico** de cupos (RNE #3), respetando los minutos previos para reservar.
- **Mis reservas**, con su estado.
- `@Scheduled` que cancela las reservas pendientes de más de 2 h y libera el cupo (RNE #2).
- Subir el registro y el panel del turista (ya hechos en la rama `joaco`).
- **Tests:** cupos insuficientes → rechaza · fuera del límite de minutos → rechaza · reserva correcta descuenta cupos · vencimiento a las 2 h libera cupos.

Persona 3 — Pagos, comentarios e integraciones

- Entidad `Pago` (monto, estado APROBADO / RECHAZADO / ERROR, fecha). Reserva–Pago 1:n (RNE #14).
- **Pasarela de pagos simulada** como servicio externo: responde aprobado, rechazado o tarda. Se llama por HTTP con **timeout**. Solo un pago aprobado confirma la reserva; si falla, la reserva sigue pendiente y se puede **reintentar**.
- Pantalla de pago (después de reservar) + botón “Reintentar pago” en Mis reservas.

- Entidad `Comentario`: valoración 1–5 + texto, una vez por reserva y solo después de la fecha (RNE #7). Editar y borrar el propio.
- API de clima (Open-Meteo, gratis, sin key) en el detalle de la actividad.
- **Tests:** aprobado → confirma · rechazado → sigue pendiente · timeout → sigue pendiente y no se cae · monto = precio × cupos · comentario antes de la fecha o repetido → rechaza.

Persona 4 — Administración y API pública

- Alta, consulta, modificación y baja de **Tipo de Actividad, Tipo de Equipamiento y Departamento** (entidad nueva), con pantalla en el panel admin. No se puede borrar uno que esté en uso.
- Verificación completa: guardar **qué admin, cuándo y el motivo** del rechazo (relación *verifica*); pedir el motivo al rechazar; historial de verificados.
- **API pública REST** (`/api/...`):
  1. listado de emprendimientos;
  2. actividades de un día por departamento o localidad;
  3. top 3 del mes por departamento;
  4. verificar emprendimiento;
  5. buscar por palabra clave.
- **Swagger** (springdoc) para documentar los endpoints.
- Limpieza: borrar `UsuarioController` (vacío), `EmprendimientoController` (hueco de seguridad) y las páginas `alta*.html` rotas; sacar H2 del `pom.xml`; base en memoria **solo para los tests**.
- **Tests:** aceptar o denegar un emprendimiento no pendiente → rechaza · borrar un tipo en uso → rechaza · top 3 · búsqueda.

Orden (dependencias)

```
Día 1    P1: Actividad + Departamento    P4: ABM de tipos y departamentos
         → se acuerdan los nombres de los campos y se suben ese mismo día
Día 2-3  P2: Reserva + Participante       P3: pasarela simulada (no depende de
nadie)
Día 4    P3: Pago conectado a Reserva + Comentario    P4: top 3 (usa Comentario)
Día 5    Todo integrado en main + tests pasando
```

Reglas para no pisarse

- Cada uno en su rama, **PR a `main` todos los días**, aunque sea chico.
- Antes de arrancar cada día: `git fetch origin` + `git merge origin/main`.
- Los archivos compartidos (`Actividad`, `Horario`, `umnatural.css`, `ControlDeAccesoInterceptor`, `LoginController`) se avisan en el grupo antes de tocarlos.

Entregables que faltan (sin asignar)

Ya hechos: actas de reunión, MER, acta fundacional, documento de visión.

- [ ] **Especificación de requerimientos con casos de uso.** Borradores en `casos-de-uso.md` y `casos-de-uso-formato-especificacion.md`; ajustarla a lo implementado.
- [ ] **Diagramas de secuencia.** Mínimo: reservar, pagar (con los caminos de error), verificar emprendimiento, alta de actividad.
- [ ] **Diagramas de flujo** (navegación y procesos principales).
- [ ] **Diagrama de arquitectura** (capas MVC, Neon, pasarela, clima, API pública).
- [ ] **Mockups** (pueden ser capturas de las pantallas reales).
- [ ] **UML de clases** del modelo (sale de las entidades).
- [ ] **Documentación de la API** (tabla de endpoints + Swagger).
- [ ] **Manuales de usuario ×3**: turista, prestador y administrador.